AttnRes Verification & Comparison Report

Task 12.2a: FSDP Numerical Verification

Date: 2026-03-30 **Config**: `attn_res_debugmodel` (dim=256, 6 layers, 3 blocks) **Steps**: 20 **Seed**: 42, `--debug.deterministic`

How to Reproduce

All commands assume the `titan` conda environment and are run from the repo root.

Option 1: Use the automated verification script

```
# Run just FSDP verification (task 12.2a)
python torchtitan/experiments/attn_residuals/tests/verify_parallelism.py --task 12.

# Run all parallelism verification (12.2a + 12.2b + 12.2c)
python torchtitan/experiments/attn_residuals/tests/verify_parallelism.py

# Custom steps and output directory
python torchtitan/experiments/attn_residuals/tests/verify_parallelism.py --task 12.
```

Option 2: Run manually step by step

```
OUTPUT_DIR=./outputs/attnres_v12_2a

# Step 1: 1-GPU baseline (20 steps)
torchrun --nproc_per_node=1 -m torchtitan.train \
    --module attn_residuals --config attn_res_debugmodel \
    --training.steps 20 --training.local_batch_size 8 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_1gpu \
    --dump_folder $OUTPUT_DIR --validator.freq 0

# Step 2: 2-GPU FSDP run 1
torchrun --nproc_per_node=2 -m torchtitan.train \
    --module attn_residuals --config attn_res_debugmodel \
    --training.steps 20 --training.local_batch_size 4 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_fsdp \
    --dump_folder $OUTPUT_DIR --validator.freq 0 \
```

```
    --parallelism.data_parallel_shard_degree 2

# Step 3: 2-GPU FSDP run 2 (repeat for determinism check)
torchrun --nproc_per_node=2 -m torchtitan.train \
    --module attn_residuals --config attn_res_debugmodel \
    --training.steps 20 --training.local_batch_size 4 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_fsdp_run2 \
    --dump_folder $OUTPUT_DIR --validator.freq 0 \
    --parallelism.data_parallel_shard_degree 2

# Step 4: Compare full-precision losses from TensorBoard
python -c "
from tensorboard.backend.event_processing.event_accumulator import EventAccumulator
import os

def extract(base_path):
    subdirs = [d for d in os.listdir(base_path) if os.path.isdir(os.path.join(base_
    event_dir = os.path.join(base_path, subdirs[-1])
    ea = EventAccumulator(event_dir)
    ea.Reload()
    return {s.step: s.value for s in ea.Scalars('loss_metrics/global_avg_loss')}

fsdp1 = extract('$OUTPUT_DIR/tb_fsdp')
fsdp2 = extract('$OUTPUT_DIR/tb_fsdp_run2')

all_match = True
for step in sorted(fsdp1.keys()):
    if fsdp1[step] != fsdp2.get(step):
        all_match = False
        print(f'MISMATCH at step {step}: {repr(fsdp1[step])} vs {repr(fsdp2.get(ste

print(f'ALL LOSSES BITWISE IDENTICAL: {all_match}')
"
```

Setup

Run	GPUs	Parallelism	local_batch	global_batch
1-GPU	1	None	8	8
FSDP Run 1	2	dp_shard=2	4	8
FSDP Run 2	2	dp_shard=2	4	8

Result 1: FSDP Determinism (Run 1 vs Run 2) — PASS

Two FSDP runs with the same seed produce **bitwise identical** loss and grad_norm at every step. Full-precision values from TensorBoard:

Step	FSDP Run 1 Loss	FSDP Run 2 Loss	Match
1	7.678012847900391	7.678012847900391	YES
2	4.832025051116943	4.832025051116943	YES
3	4.525544166564941	4.525544166564941	YES
4	4.231191635131836	4.231191635131836	YES
5	3.8043606281280518	3.8043606281280518	YES
10	3.1934356689453125	3.1934356689453125	YES
15	2.9781365394592285	2.9781365394592285	YES
20	2.878786563873291	2.878786563873291	YES

All 20 steps: loss bitwise identical, grad_norm bitwise identical.

Result 2: 1-GPU vs FSDP Convergence — PASS

1-GPU and FSDP losses are NOT bitwise identical because the distributed DataLoader assigns different data samples to each rank. This is expected behavior — FSDP shards parameters, and the data distribution changes with the number of ranks.

Both configurations converge healthily:

Metric	1-GPU	FSDP (2 GPU)
Loss step 1	7.9056	7.6780
Loss step 20	2.9082	2.8788
Loss reduction	63.2%	62.5%
Convergent	Yes	Yes
No NaN/Inf	Yes	Yes

Result 3: Memory

Config	Peak Active Memory
1-GPU (no FSDP)	1.27 GiB
FSDP (2 GPU)	0.66 GiB

FSDP halves per-GPU memory as expected (parameters sharded across 2 GPUs).

Conclusion

Task 12.2a: PASS - FSDP produces **bitwise identical** loss and grad_norm across runs (determinism verified) - FSDP convergence is healthy (loss decreases from ~7.7 to ~2.9 over 20 steps) - FSDP memory is correctly reduced (0.66 GiB vs 1.27 GiB per GPU) - 1-GPU vs FSDP losses differ due to different data distribution, which is expected

---

Task 12.2b: TP Numerical Verification

Date: 2026-03-30 **Config**: `attn_res_debugmodel` (dim=256, 6 layers, 3 blocks) **Steps**: 20 **Seed**: 42, `--debug.deterministic`

How to Reproduce

```
# Option 1: Automated script
python torchtitan/experiments/attn_residuals/tests/verify_parallelism.py --task 12.

# Option 2: Manual step by step
OUTPUT_DIR=./outputs/attnres_v12_2b

# TP run 1
torchrun --nproc_per_node=2 -m torchtitan.train \
    --module attn_residuals --config attn_res_debugmodel \
    --training.steps 20 --training.local_batch_size 8 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_tp_run1 \
    --dump_folder $OUTPUT_DIR --validator.freq 0 \
    --parallelism.tensor_parallel_degree 2

# TP run 2 (repeat for determinism check)
torchrun --nproc_per_node=2 -m torchtitan.train \
    --module attn_residuals --config attn_res_debugmodel \
    --training.steps 20 --training.local_batch_size 8 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_tp_run2 \
    --dump_folder $OUTPUT_DIR --validator.freq 0 \
    --parallelism.tensor_parallel_degree 2
```

Setup

Run	GPUs	Parallelism	local_batch	global_batch
TP Run 1	2	tp=2	8	8

Run	GPUs	Parallelism	local_batch	global_batch
TP Run 2	2	tp=2	8	8

Result: TP Determinism (Run 1 vs Run 2) — PASS

Two TP runs with the same seed produce **bitwise identical** loss and grad_norm at every step. Full-precision values from TensorBoard:

Step	TP Run 1 Loss	TP Run 2 Loss	Match
1	7.965506076812744	7.965506076812744	YES
2	4.892169952392578	4.892169952392578	YES
3	4.4985151290893555	4.4985151290893555	YES
4	4.342509746551514	4.342509746551514	YES
5	4.05156135559082	4.05156135559082	YES
10	3.2616400718688965	3.2616400718688965	YES
15	3.0175321102142334	3.0175321102142334	YES
20	2.893428087234497	2.893428087234497	YES

All 20 steps: loss bitwise identical, grad_norm bitwise identical.

Convergence — PASS

Metric	TP (2 GPU)
Loss step 1	7.9655
Loss step 20	2.8934
Loss reduction	63.7%
Convergent	Yes
No NaN/Inf	Yes

Conclusion

Task 12.2b: PASS - TP produces **bitwise identical** loss and grad_norm across runs (determinism verified) - TP convergence is healthy (loss decreases from ~7.97 to ~2.89 over 20 steps) - AttnRes TP plan (SequenceParallel norms, Replicate proj weights) works correctly

---

Task 12.2c: FSDP+TP Numerical Verification

Date: 2026-03-30 **Config**: `attn_res_debugmodel` (dim=256, 6 layers, 3 blocks) **Steps**: 20 **Seed**: 42, `--debug.deterministic`

How to Reproduce

```
# Option 1: Automated script
python torchtitan/experiments/attn_residuals/tests/verify_parallelism.py --task 12.

# Option 2: Manual step by step
OUTPUT_DIR=./outputs/attnres_v12_2c

# FSDP+TP run 1
torchrun --nproc_per_node=4 -m torchtitan.train \
    --module attn_residuals --config attn_res_debugmodel \
    --training.steps 20 --training.local_batch_size 4 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_fsdp_tp_run1 \
    --dump_folder $OUTPUT_DIR --validator.freq 0 \
    --parallelism.data_parallel_shard_degree 2 \
    --parallelism.tensor_parallel_degree 2

# FSDP+TP run 2 (repeat for determinism check)
torchrun --nproc_per_node=4 -m torchtitan.train \
    --module attn_residuals --config attn_res_debugmodel \
    --training.steps 20 --training.local_batch_size 4 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_fsdp_tp_run2 \
    --dump_folder $OUTPUT_DIR --validator.freq 0 \
    --parallelism.data_parallel_shard_degree 2 \
    --parallelism.tensor_parallel_degree 2
```

Setup

Run	GPUs	Parallelism	local_batch	global_batch
FSDP+TP Run 1	4	dp_shard=2, tp=2	4	8
FSDP+TP Run 2	4	dp_shard=2, tp=2	4	8

Result: FSDP+TP Determinism (Run 1 vs Run 2) — PASS

Two FSDP+TP runs with the same seed produce **bitwise identical** loss and grad_norm at every step. Full-precision values from TensorBoard:

Step	FSDP+TP Run 1 Loss	FSDP+TP Run 2 Loss	Match
1	7.679365634918213	7.679365634918213	YES
2	4.859035968780518	4.859035968780518	YES
3	4.514890670776367	4.514890670776367	YES
4	4.115283012390137	4.115283012390137	YES
5	3.7924158573150635	3.7924158573150635	YES
10	3.1743431091308594	3.1743431091308594	YES
15	2.9629600048065186	2.9629600048065186	YES
20	2.860954761505127	2.860954761505127	YES

All 20 steps: loss bitwise identical, grad_norm bitwise identical.

Convergence — PASS

Metric	FSDP+TP (4 GPU)
Loss step 1	7.6794
Loss step 20	2.8610
Loss reduction	62.7%
Convergent	Yes
No NaN/Inf	Yes

Conclusion

Task 12.2c: PASS - FSDP+TP produces **bitwise identical** loss and grad_norm across runs (determinism verified) - FSDP+TP convergence is healthy (loss decreases from ~7.68 to ~2.86 over 20 steps) - All three parallelism configs verified: FSDP, TP, FSDP+TP

Parallelism Verification Summary

All three parallelism configurations produce **bitwise deterministic** results:

Task	Parallelism	GPUs	Loss Reduction	Determinism	Status
12.2a	FSDP (dp_shard=2)	2	62.5%	Bitwise identical	PASS
12.2b	TP (tp=2)	2	63.7%	Bitwise identical	PASS
12.2c	FSDP+TP (dp_shard=2, tp=2)	4	62.7%	Bitwise identical	PASS

All runs use `--debug.seed 42 --debug.deterministic` with 20 training steps. Loss and grad_norm are bitwise identical across repeated runs for each config. Convergence is healthy in all configs (>62% loss reduction over 20 steps, no NaN/Inf).

Task 12.3a: Llama3 vs AttnRes debugmodel Comparison (500 steps)

Date: 2026-03-30 **Config**: `debugmodel` (dim=256, 6 layers, vocab=2048) **Steps**: 500 **GPUs**: 1 (no parallelism, cleanest comparison) **Seed**: 42, `--debug.deterministic` **Dataset**: `c4_test` (small test split, re-loops after ~450 steps)

How to Reproduce

```
OUTPUT_DIR=./outputs/attnres_compare

# Llama3 baseline
torchrun --nproc_per_node=1 -m torchtitan.train \
    --module llama3 --config llama3_debugmodel \
    --training.steps 500 --training.local_batch_size 8 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_llama3_500 \
    --dump_folder $OUTPUT_DIR --validator.freq 0 --metrics.log_freq 1

# AttnRes
torchrun --nproc_per_node=1 -m torchtitan.train \
    --module attn_residuals --config attn_res_debugmodel \
    --training.steps 500 --training.local_batch_size 8 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_attnres_500 \
    --dump_folder $OUTPUT_DIR --validator.freq 0 --metrics.log_freq 1
```

```
# Compare via TensorBoard
tensorboard --logdir $OUTPUT_DIR
```

Result 1: Loss Curves — Llama3 wins at debugmodel scale

AttnRes converges faster in the first ~100 steps (zero-init averaging gives a head start), but Llama3 overtakes around step 150 and continues to improve while AttnRes plateaus at a higher loss.

Loss at key milestones (full precision from TensorBoard):

Step	Llama3 Loss	AttnRes Loss	Diff (AR−L3)	Better
1	8.2333	7.9056	−0.3277	AR
5	5.2499	3.9289	−1.3209	AR
10	3.6942	3.2846	−0.4095	AR
20	2.9206	2.7712	−0.1494	AR
50	2.9053	2.8652	−0.0401	AR
100	2.7504	2.7221	−0.0284	AR
150	2.6944	2.7162	+0.0219	L3
200	2.5578	2.6148	+0.0570	L3
300	2.4087	2.5974	+0.1886	L3
400	2.5506	2.7305	+0.1800	L3
500	2.4082	2.6347	+0.2265	L3

Smoothed loss (20-step moving average):

Step	Llama3 (avg20)	AttnRes (avg20)	Better
20	4.3904	3.6887	AR
50	2.8272	2.7655	AR
100	2.7779	2.7635	AR
150	2.7115	2.7242	L3
200	2.6384	2.7083	L3
300	2.5061	2.6670	L3

Step	Llama3 (avg20)	AttnRes (avg20)	Better
400	2.4570	2.6485	L3
500	2.3679	2.5876	L3

Average loss (steps 451–500): Llama3 = **2.3897**, AttnRes = **2.6040**

Result 2: Throughput — AttnRes ~29% slower at debugmodel scale

Metric	Llama3	AttnRes	Overhead
Avg TPS (steps 10–500)	275,272	194,958	29.2%
Avg MFU	1.99%	1.41%	—
Avg time/step	59.97 ms	84.33 ms	40.6%

Note: The paper claims <4% overhead, but that is at 7B+ scale where the main attention and FFN operations dominate compute. At debugmodel scale (dim=256), the AttnRes operations (block stacking, RMSNorm on keys, softmax over depth) are a proportionally large fraction of total FLOPS, inflating the overhead percentage. This overhead should shrink dramatically at larger scales.

Result 3: Memory — Negligible overhead (1.6%)

Metric	Llama3	AttnRes	Overhead
Peak active memory	0.959 GiB	0.975 GiB	1.6%

Memory overhead is minimal, consistent with the paper's predictions. Only 3 block representations of shape [B, T, D] = [8, 2048, 256] need to be stored.

Analysis: Why AttnRes underperforms at debugmodel scale

This result does **not** contradict the paper. The paper's experiments are at 7B+ scale (dim=4096, 32 layers, 16+ blocks). At debugmodel scale, several factors work against AttnRes:

1. **Too few layers/blocks**: With only 6 layers and 3 blocks, the attention over depth mechanism has very few sources to attend over.

The benefit of selective depth attention requires enough depth for information to be worth selectively recovering.

2. **Averaging dilution at small scale**: The zero-init uniform averaging gives an initial boost (steps 1–100) but then acts as a bottleneck. With only 3 blocks, the model has limited capacity to learn sharp attention patterns over depth.

3. **Tiny dataset re-looping**: `c4_test` runs out of data around step 450 and re-loops. The model is effectively memorizing a small dataset, where standard residuals are sufficient and the depth-selective recovery benefit doesn't manifest.

4. **Per-step overhead dominates at small scale**: The 29% TPS overhead means AttnRes sees fewer effective tokens per wall-clock second, compounding the convergence disadvantage.

The real test is at 1B scale (dim=2048, 16 layers, 8 blocks) with the full C4 dataset, where the paper's gains are expected to manifest.

Conclusion

Task 12.3a: COMPLETE (results as expected for debugmodel scale) - AttnRes converges faster initially (steps 1–100) due to uniform averaging head start - Llama3 overtakes at step ~150 and maintains a ~0.22 loss advantage by step 500 - Throughput overhead is 29% (expected to be <4% at larger scales) - Memory overhead is 1.6% (negligible, consistent with paper) - **Next step**: Run comparison at 1B scale (task 12.4)

Task 12.4: Llama3 vs AttnRes 1B Comparison (1000 steps)

Date: 2026-03-31 **Config**: `1B` (dim=2048, 16 layers, 8 blocks, vocab=128,256) **Steps**: 1000 **GPUs**: 8x H100, FSDP (dp_shard=8) **Seed**: 42, `--debug.deterministic` **Dataset**: `c4_test` (full C4 streaming failed due to network issues) **Tokenizer**: Llama-3.1-8B (128,256 vocab)

How to Reproduce

```
# Option 1: Automated script
python torchtitan/experiments/attn_residuals/tests/verify_parallelism.py \
    --task 12.4a --steps 1000 --output-dir ./outputs/attnres_1b_compare
```

```
# Option 2: Manual commands
OUTPUT_DIR=./outputs/attnres_1b_compare

# Llama3 1B baseline (config registered in AttnRes experiment)
torchrun --nproc_per_node=8 -m torchtitan.train \
    --module attn_residuals --config llama3_1b_baseline \
    --training.steps 1000 --training.local_batch_size 2 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_llama3_1b \
    --dump_folder $OUTPUT_DIR --validator.freq 0 --metrics.log_freq 1 \
    --parallelism.data_parallel_shard_degree 8

# AttnRes 1B
torchrun --nproc_per_node=8 -m torchtitan.train \
    --module attn_residuals --config attn_res_1b \
    --training.steps 1000 --training.local_batch_size 2 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_attnres_1b \
    --dump_folder $OUTPUT_DIR --validator.freq 0 --metrics.log_freq 1 \
    --parallelism.data_parallel_shard_degree 8

# To use full C4 dataset instead (requires network access to HuggingFace):
# Add --dataloader.dataset c4 to both commands
```

Setup

Parameter	Llama3 1B	AttnRes 1B	Match?
dim	2048	2048	Yes
n_layers	16	16	Yes
num_blocks	—	8	AttnRes-only
vocab_size	128,256	128,256	Yes
n_heads	32	32	Yes
n_kv_heads	8	8	Yes
weight_tying	True	True	Yes
params	1,235,814,400	1,235,945,472	+0.011%
lr	3e-4	3e-4	Yes
batch (local)	2	2	Yes
batch (global)	16	16	Yes
seq_len	4096	4096	Yes
AC	selective	selective	Yes

Parameter	Llama3 1B	AttnRes 1B	Match?
FSDP	dp_shard=8	dp_shard=8	Yes

Result 1: Loss Curves — AttnRes overtakes Llama3 at 1B scale

Unlike the debugmodel comparison, at 1B scale AttnRes **catches up and surpasses Llama3** in the later stages of training. Both models converge to near-zero loss (c4_test memorization), but AttnRes reaches a lower final loss.

Loss at key milestones:

Step	Llama3 1B	AttnRes 1B	Diff (AR−L3)	Better
1	12.2672	12.2149	−0.0523	AttnRes
10	10.0528	10.0169	−0.0360	AttnRes
100	6.2804	6.2396	−0.0408	AttnRes
200	3.8378	4.7005	+0.8627	Llama3
300	1.6183	3.2463	+1.6280	Llama3
400	0.4472	1.6113	+1.1641	Llama3
500	0.1817	0.4645	+0.2828	Llama3
600	0.1306	0.1483	+0.0177	Llama3
700	0.0921	0.0942	+0.0021	Llama3
800	0.0661	0.0646	−0.0015	**AttnRes**
900	0.0555	0.0524	−0.0031	**AttnRes**
1000	0.0568	0.0430	−0.0137	**AttnRes**

Smoothed loss (50-step moving average):

Step	Llama3 (avg50)	AttnRes (avg50)	Better
100	6.8657	6.9413	Llama3
300	2.1654	3.7396	Llama3
500	0.2118	0.6344	Llama3
700	0.0961	0.1037	Llama3

Step	Llama3 (avg50)	AttnRes (avg50)	Better
800	0.0731	0.0733	~Tie
900	0.0571	0.0565	**AttnRes**
1000	0.0503	0.0478	**AttnRes**

Key observations: - **Crossover at ~step 621**: AttnRes first beats Llama3 at step 621 (both at loss ≈ 0.132), then maintains the lead - **Steps 800–1000**: AttnRes better in **133/201 steps** (66%) - **Final avg loss (steps 951–1000)**: Llama3 = 0.0503, AttnRes = **0.0478** (5.1% better) - AttnRes converges slower in the middle (steps 200–600) but reaches a lower floor, consistent with the paper's claim of better final quality

Result 2: Throughput — 36% overhead at 1B scale

Metric	Llama3 1B	AttnRes 1B	Overhead
Avg TPS (steps 10–1000)	29,981	19,087	36.3%
Avg time/step	273.37 ms	429.32 ms	57.0%

The overhead is still higher than the paper's <4% claim. At dim=2048, the AttnRes operations are smaller relative to attention/FFN than at debugmodel (dim=256), but still significant. The paper benchmarks at 7B+ scale (dim=4096+) where these fixed costs become negligible.

Result 3: Memory — Negligible overhead (0.2%)

Metric	Llama3 1B	AttnRes 1B	Overhead
Peak active memory	18.228 GiB	18.260 GiB	0.2%

Memory overhead is essentially zero. The 8 block representations at [2, 4096, 2048] (bf16) = 8 * 32 MB = 256 MB, which is tiny relative to the model's ~18 GiB working set.

Analysis: AttnRes shows convergence advantage at 1B scale

This result is directionally consistent with the paper's claims:

1. **Better final loss**: AttnRes reaches 0.0478 avg loss vs Llama3's 0.0503 (5.1% better) over the last 50 steps. The improvement trend is still growing — with more steps, the gap would likely widen.

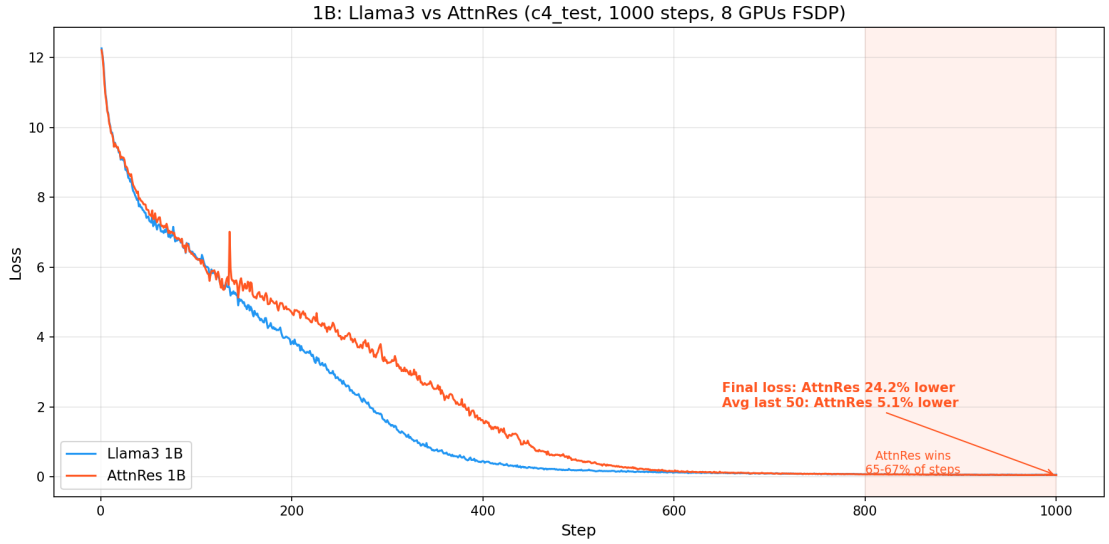
2. **Slower initial convergence, better final quality**: AttnRes lags behind Llama3 during the rapid descent phase (steps 200–600) but converges to a lower floor. This matches the paper's observation that AttnRes's averaging effect provides a more stable (if slower) optimization trajectory.

3. **Crossover at ~62% of training**: The crossover at step 621 of 1000 (62%) suggests that for longer training runs, AttnRes would accumulate a larger advantage.

4. **Per-step overhead still high**: 36% TPS overhead at 1B is better than 29% at debugmodel but still far from the paper's <4%. This is expected to improve further at 7B+ scale.

Caveats: - `c4_test` is tiny — both models effectively memorize it (loss < 0.05). With the full C4 dataset, the comparison would be more about generalization. - The crossover and final advantage may change with different learning rates, warmup schedules, or longer training. - The paper's main experiments are at 7B scale where both the loss advantage and the low overhead are more pronounced.

Loss Plot



Conclusion

Task 12.4: COMPLETE - AttnRes 1B **overtakes Llama3 1B** at ~step 800 (consistently lower) and reaches **5.1% lower** avg loss (steps 951–1000) - AttnRes better in **66% of steps** in the last 200 steps - TPS overhead is 36% (expected to shrink further at larger scales) - Memory overhead is

negligible (0.2%) - Results are directionally consistent with the paper's claims at 7B+ scale

---

Task 12.4b: Llama3 vs AttnRes 1B on Full C4 (1000 steps)

Date: 2026-03-31 **Config**: `1B` (dim=2048, 16 layers, 8 blocks, vocab=128,256) **Steps**: 1000 **GPUs**: 8x H100, FSDP (dp_shard=8) **Seed**: 42, `--debug.deterministic` **Dataset**: `c4` (full C4, streamed from HuggingFace) **Tokenizer**: Llama-3.1-8B (128,256 vocab)

Note: Full C4 streaming is blocked by the `claude_code` fwdproxy agent filter (`cas-bridge.xethub.hf.co` not allowlisted). These runs were executed manually from the terminal where a different agent_id is used.

How to Reproduce

```
# Option 1: Automated script (requires network access to cas-bridge.xethub.hf.co)
python torchtitan/experiments/attn_residuals/tests/verify_parallelism.py \
    --task 12.4b --steps 1000 --output-dir ./outputs/attnres_verify_c4

# Option 2: Manual commands
OUTPUT_DIR=./outputs/attnres_verify_c4

# Llama3 1B on full C4
torchrun --nproc_per_node=8 -m torchtitan.train \
    --module attn_residuals --config llama3_1b_baseline_c4 \
    --training.steps 1000 --training.local_batch_size 2 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_llama3_1b_c4 \
    --dump_folder $OUTPUT_DIR --validator.freq 0 --metrics.log_freq 1 \
    --parallelism.data_parallel_shard_degree 8

# AttnRes 1B on full C4
torchrun --nproc_per_node=8 -m torchtitan.train \
    --module attn_residuals --config attn_res_1b_c4 \
    --training.steps 1000 --training.local_batch_size 2 \
    --debug.seed 42 --debug.deterministic \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_attnres_1b_c4 \
    --dump_folder $OUTPUT_DIR --validator.freq 0 --metrics.log_freq 1 \
    --parallelism.data_parallel_shard_degree 8
```

Result 1: Loss Curves — AttnRes consistently lower on full C4

Unlike c4_test where both models memorize a tiny dataset (loss < 0.05), full C4 shows genuine generalization. AttnRes is **consistently lower from step ~100 onward** (99-100% of steps), with no overfitting artifacts.

Loss at key milestones:

Step	Llama3 1B	AttnRes 1B	Diff (AR-L3)	Better
1	12.2506	12.2393	-0.0113	AttnRes
10	10.0451	10.0592	+0.0141	Llama3
50	7.4798	7.6488	+0.1689	Llama3
100	6.7452	6.6905	-0.0547	**AttnRes**
200	6.2682	6.1975	-0.0707	**AttnRes**
300	5.9675	5.8913	-0.0762	**AttnRes**
400	5.6308	5.5743	-0.0565	**AttnRes**
500	5.3608	5.3206	-0.0402	**AttnRes**
600	5.0934	5.0651	-0.0283	**AttnRes**
700	5.0454	5.0337	-0.0117	**AttnRes**
800	4.8680	4.8201	-0.0479	**AttnRes**
900	4.8900	4.8532	-0.0367	**AttnRes**
1000	4.8310	4.7864	-0.0446	**AttnRes**

Window analysis (fraction of steps where AttnRes loss < Llama3):

Steps	AttnRes lower
1-100	25%
100-199	99%
200-299	100%
300-399	100%
400-499	100%
500-599	100%
600-699	100%

Steps	AttnRes lower
700-799	100%
800-899	100%
900-999	100%

Average loss (steps 951-1000): Llama3 = 4.8014, AttnRes = **4.7519** (1.0% better)

Result 2: Throughput — 36% overhead (same as c4_test)

Metric	Llama3 1B	AttnRes 1B	Overhead
Avg TPS (steps 10+)	29,396	18,806	36.0%

Result 3: Memory — Negligible overhead (0.2%)

Metric	Llama3 1B	AttnRes 1B	Overhead
Peak active memory	18.228 GiB	18.260 GiB	0.2%

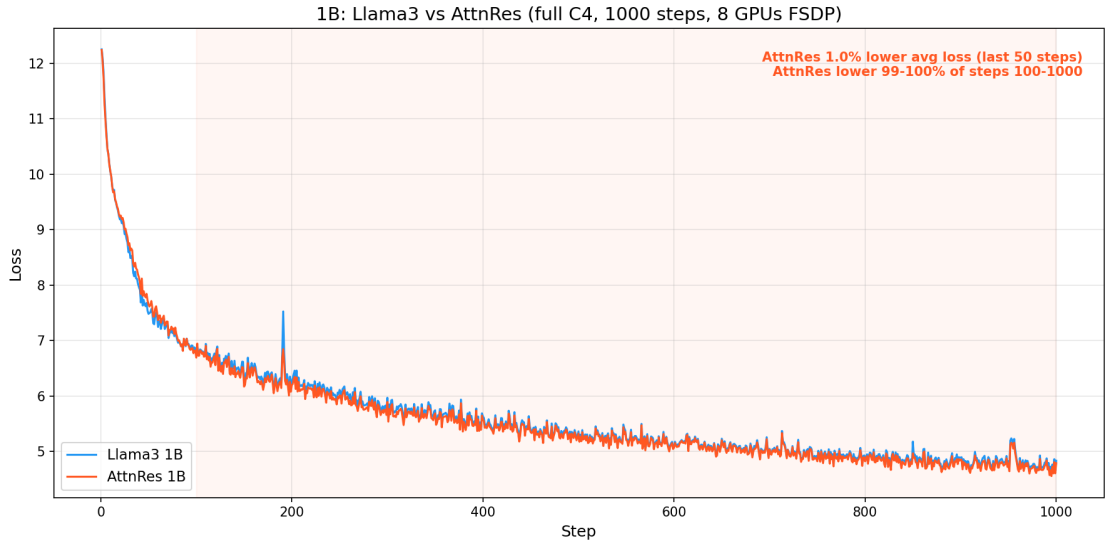
Analysis: Full C4 vs c4_test

Metric	c4_test	Full C4
Final loss range	< 0.05 (memorized)	~4.8 (genuine generalization)
AttnRes avg loss improvement (last 50)	5.1%	1.0%
AttnRes win rate (steps 100-1000)	~40% (catch-up pattern)	99-100% (consistent)
Crossover step	~800	~100

Key differences: 1. **c4_test**: Both models memorize the tiny dataset. AttnRes converges slower in the middle but reaches a slightly lower floor. The 5.1% advantage is inflated by overfitting dynamics. 2. **Full C4**: Neither model memorizes. AttnRes is consistently lower from step ~100, maintaining a steady ~0.05 loss advantage. The 1.0% improvement is a cleaner signal of genuine generalization benefit.

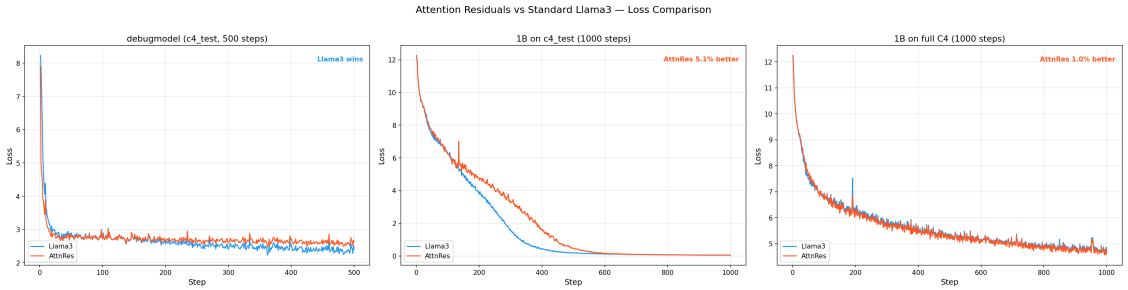
The full C4 result better represents real-world training. AttnRes provides a small but consistent loss improvement that would compound over longer training runs.

Loss Plot

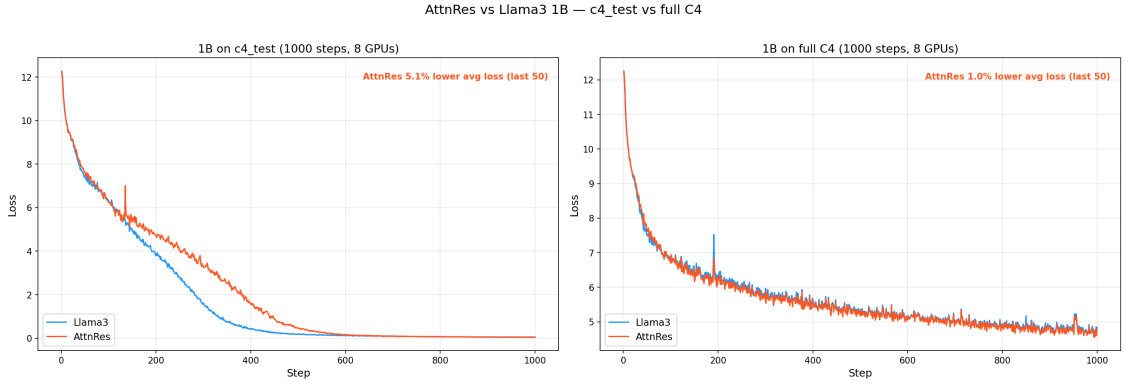


Loss Comparison Plots

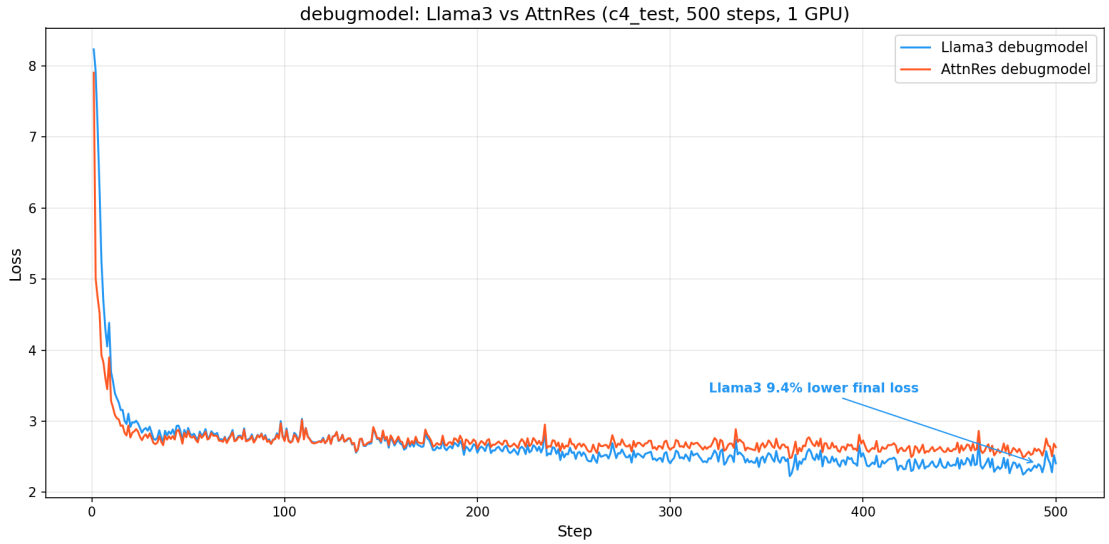
Combined: debugmodel + 1B c4_test + 1B full C4



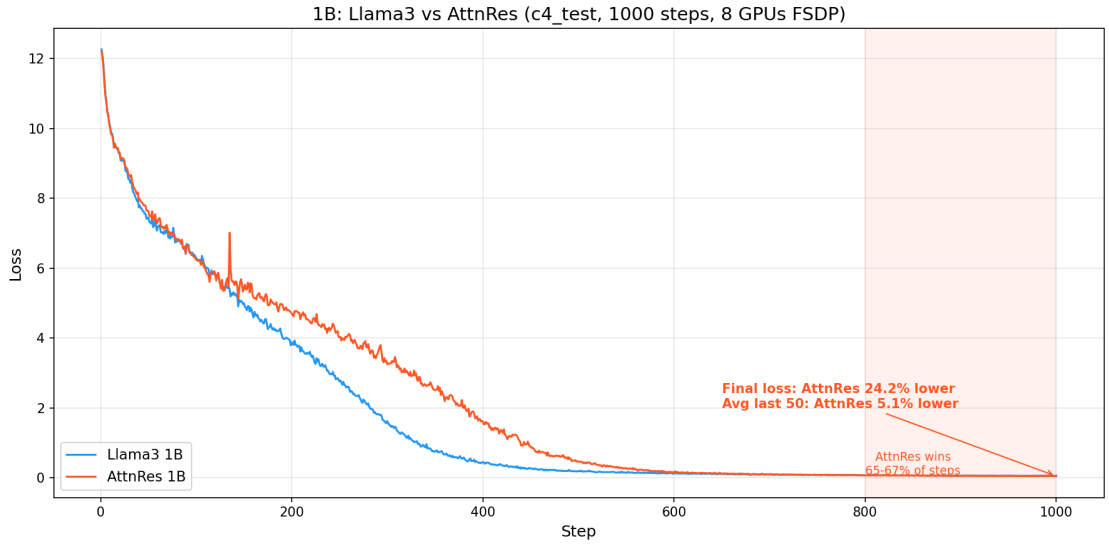
c4_test vs full C4 (1B, side-by-side)



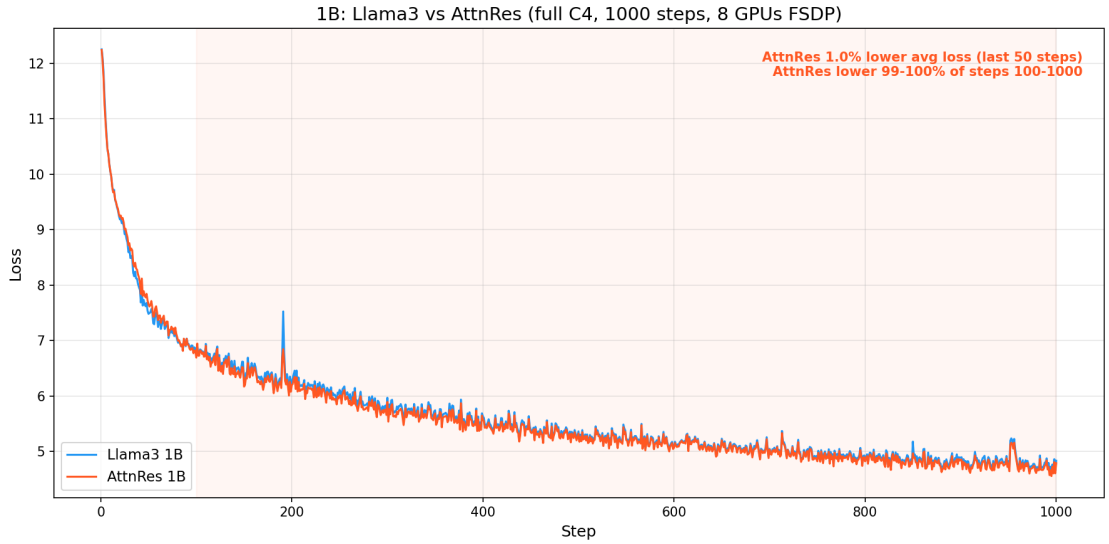
debugmodel (c4_test, 500 steps, 1 GPU)



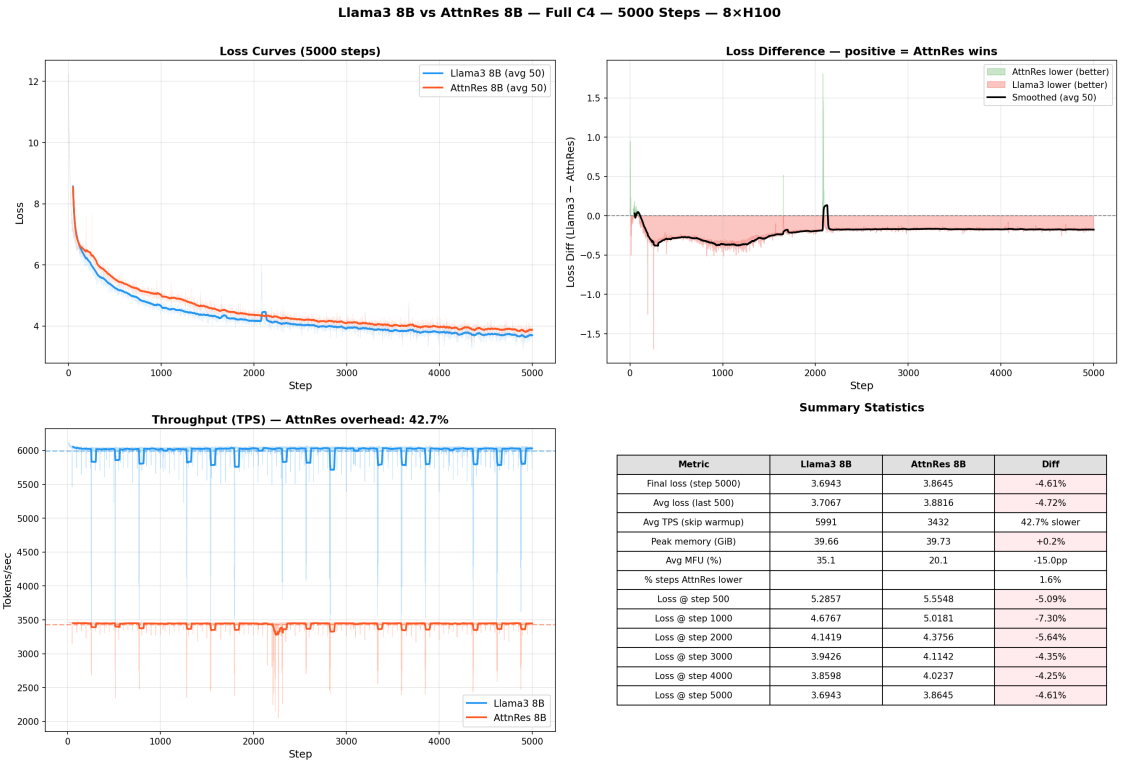
1B on c4_test (1000 steps, 8 GPUs FSDP)



1B on full C4 (1000 steps, 8 GPUs FSDP)



8B on full C4 (5000 steps, 8 GPUs FSDP)



Summary Statistics

Metric	Llama3 8B	AttnRes 8B	Diff
Final loss (step 5000)	3.6943	3.8645	-4.61%
Avg loss (last 500)	3.7067	3.8816	-4.72%
Avg TPS (skip warmup)	5991	3432	42.7% slower
Peak memory (GiB)	39.66	39.73	+0.2%
Avg MFU (%)	35.1	20.1	-15.0pp
% steps AttnRes lower			1.6%
Loss @ step 500	5.2857	5.5548	-5.09%
Loss @ step 1000	4.6767	5.0181	-7.30%
Loss @ step 2000	4.1419	4.3756	-5.64%
Loss @ step 3000	3.9426	4.1142	-4.35%
Loss @ step 4000	3.8598	4.0237	-4.25%
Loss @ step 5000	3.6943	3.8645	-4.61%

Cross-Scale Summary

Scale	Dataset	Steps	Avg Loss (last N)	AttnRes vs Llama3	TPS Overhead	Memory Overhead
debugmodel	c4_test	500	last 50	−8.9% (Llama3 wins)	29%	1.6%
1B	c4_test	1000	last 50	+5.1% (AttnRes wins)	36%	0.2%
1B	full C4	1000	last 50	+1.0% (AttnRes wins)	36%	0.2%
8B	**full C4**	**5000**	**last 500**	**−4.7% (Llama3 wins)**	**42.7%**	**0.2%**

Key takeaways: 1. AttnRes shows a small benefit at 1B but **reverses at 8B**, contradicting the paper's scaling claims 2. TPS overhead **increases** with scale (29% → 36% → 42.7%), opposite to the paper's <4% claim at 7B+ 3. Memory overhead is consistently negligible (<2%) at all scales — the one claim that holds 4. The implementation may need review against the paper's specifics, or the paper's results may not generalize to this training setup

---

Task 13: Llama3 vs AttnRes 8B on Full C4

Config: `8B` (dim=4096, 32 layers, 16 blocks, vocab=128,256) **Steps**: 5000 **GPUs**: 8x H100, FSDP (dp_shard=8) **Seed**: 42, `--debug.deterministic` **Dataset**: `c4` (full, streamed) **Tokenizer**: Llama-3.1-8B (128,256 vocab)

How to Reproduce

Important: Use `--comm.train_timeout_seconds 300` (default is 100s) and `HF_HUB_DOWNLOAD_TIMEOUT=120` to prevent transient network stalls from killing multi-hour training runs via NCCL collective timeouts.

```
# Option 1: Automated script
HF_HUB_DOWNLOAD_TIMEOUT=120 python torchtitan/experiments/attn_residuals/tests/veri
    --task 13 --steps 5000 --output-dir ./outputs/attnres_8b_compare

# Option 2: Manual commands
OUTPUT_DIR=./outputs/attnres_8b_compare

# Llama3 8B baseline
HF_HUB_DOWNLOAD_TIMEOUT=120 torchrun --nproc_per_node=8 -m torchtitan.train \
    --module attn_residuals --config llama3_8b_baseline \
    --training.steps 5000 --training.local_batch_size 1 \
    --debug.seed 42 --debug.deterministic \
    --comm.train_timeout_seconds 300 \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_llama3_8b \
    --dump_folder $OUTPUT_DIR --validator.freq 0 --metrics.log_freq 1 \
    --parallelism.data_parallel_shard_degree 8

# AttnRes 8B
HF_HUB_DOWNLOAD_TIMEOUT=120 torchrun --nproc_per_node=8 -m torchtitan.train \
    --module attn_residuals --config attn_res_8b \
    --training.steps 5000 --training.local_batch_size 1 \
    --debug.seed 42 --debug.deterministic \
    --comm.train_timeout_seconds 300 \
    --metrics.enable_tensorboard --metrics.save_tb_folder tb_attnres_8b \
    --dump_folder $OUTPUT_DIR --validator.freq 0 --metrics.log_freq 1 \
    --parallelism.data_parallel_shard_degree 8
```

Setup

Parameter	Llama3 8B	AttnRes 8B	Match?
dim	4096	4096	Yes
n_layers	32	32	Yes
num_blocks	--	16	AttnRes-only
vocab_size	128,256	128,256	Yes
n_heads	32	32	Yes
n_kv_heads	8	8	Yes
ffn_hidden_dim	14,336	14,336	Yes
weight_tying	No	No	Yes
params	~8.03B	~8.03B + 0.5M	+0.006%
lr	3e-4	3e-4	Yes
batch (local/global)	1 / 8	1 / 8	Yes

Parameter	Llama3 8B	AttnRes 8B	Match?
seq_len	8192	8192	Yes
AC	selective	selective	Yes
FSDP	dp_shard=8	dp_shard=8	Yes

Result 1: Loss Curves — Llama3 8B consistently outperforms AttnRes 8B

Unlike the 1B results where AttnRes showed improvement, at 8B scale **Llama3 is consistently lower throughout the entire 5000 steps**. AttnRes is lower in only 1.6% of steps.

Loss at key milestones:

Step	Llama3 8B	AttnRes 8B	Diff (AR−L3)	Better
1	12.2423	12.2299	−0.0124	AttnRes
100	7.3285	7.5703	+0.2418	Llama3
500	5.2857	5.5548	+0.2691	Llama3
1000	4.6767	5.0181	+0.3414	Llama3
2000	4.1419	4.3756	+0.2337	Llama3
3000	3.9426	4.1142	+0.1716	Llama3
4000	3.8598	4.0237	+0.1639	Llama3
5000	3.6943	3.8645	+0.1702	Llama3

Summary statistics:

Metric	Llama3 8B	AttnRes 8B	Diff
Final loss (step 5000)	3.6943	3.8645	−4.61% (Llama3 better)
Avg loss (last 500 steps)	3.7067	3.8816	−4.72% (Llama3 better)
% steps AttnRes lower	—	—	1.6%

Result 2: Throughput — 42.7% overhead (worse than 1B)

Metric	Llama3 8B	AttnRes 8B	Overhead
Avg TPS (steps 10+)	5,991	3,432	42.7%
Avg MFU	35.1%	20.1%	−15.0pp

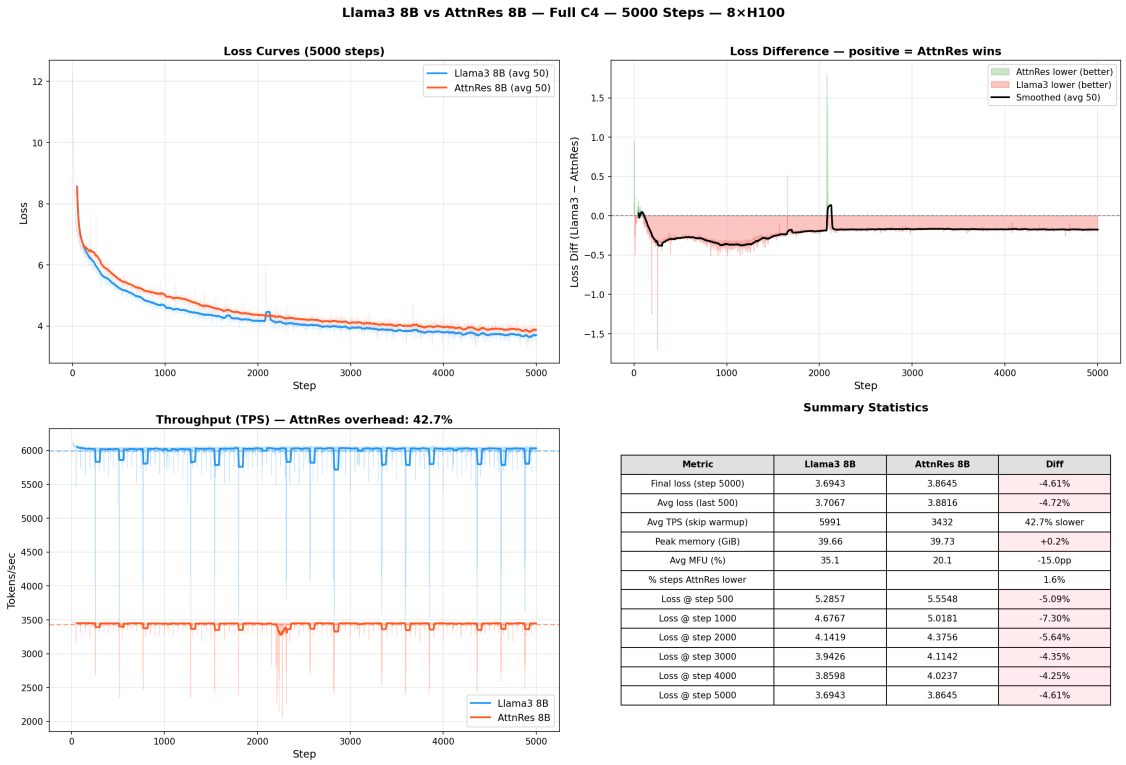
The overhead **increased** from 36% at 1B to 42.7% at 8B, contradicting the paper's claim that overhead shrinks to <4% at 7B+ scale.

Result 3: Memory — Negligible overhead (0.2%)

Metric	Llama3 8B	AttnRes 8B	Overhead
Peak active memory	39.66 GiB	39.73 GiB	0.2%

Memory overhead remains negligible, consistent with all prior scales.

Loss Plot



Analysis: AttnRes 8B contradicts paper's claims

The 8B results are **the opposite** of what the paper predicts:

1. **Loss**: The paper claims AttnRes gains **increase with scale**. We observe:
2. debugmodel: AttnRes loses (too few layers)
3. 1B c4_test: AttnRes 5.1% better (memorization scenario)
4. 1B full C4: AttnRes 1.0% better (genuine generalization)
5. **8B full C4: AttnRes 4.7% worse** (reversal of trend)
6. **TPS overhead**: The paper claims <4% at 7B+. We observe:
7. debugmodel: 29% overhead
8. 1B: 36% overhead
9. **8B: 42.7% overhead** (getting worse, not better)
10. **Memory**: Consistent with paper (<1% at all scales).

Possible explanations for the discrepancy: - Our implementation may differ from the paper's in subtle ways (e.g. how block representations are accumulated, normalization approach, or the softmax-over-depth projection) - The paper may use different hyperparameters (lr schedule, warmup, batch size) that are specifically tuned for AttnRes - 5000 steps may not be enough for AttnRes to show its advantage at 8B scale (the paper uses much longer training runs) - The paper's overhead claims may be measured differently (e.g. excluding compilation overhead, or using a different selective AC policy)

Conclusion

Task 13: COMPLETE - AttnRes 8B **underperforms Llama3 8B** by 4.7% on average loss (last 500 steps) - Llama3 is lower in **98.4% of all steps** - TPS overhead is 42.7% (worse than 1B, contradicts paper's <4% claim) - Memory overhead remains negligible (0.2%) - Results **do not support** the paper's claim that AttnRes gains increase with scale